Notes on ML Models

1 Principal Component Analysis

PCA algorithm is to find a **linear and orthogonal** projection of the high dimensional data $x \in \mathbb{R}^D$, to a low dimensional subspace $z \in \mathbb{R}^L$, such that the low dimensional representation is a "good approximation" to the original data. If we project or encode $mathbfx$ to get $\mathbf{z} = \mathbf{W^T x}$, and then unproject or decode $\mathbf{z}$ to get $\widehat{\mathbf{x}} = \mathbf{Wz}$, then we want $\mathbf{x}$ to be close to $\mathbf{x}$ in ℓ_2 distance. In particular, we can define the following **reconstruction error**:

$$\mathcal{L}(\mathbf{W}) \triangleq \frac{1}{N}\sum_{n=1}^{N} \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; \mathbf{W}); \mathbf{W})\|_2^2$$

PCA algorithm states that this objective can be minimized by setting $\mathbf{W} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the the L eigenvectors with lages eigenvalues of the empirical covariance matrix

$$\hat{\mathbf{\Sigma}} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \overline{\boldsymbol{x}})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})^\mathsf{T} = \frac{1}{N}\mathbf{X}_c^\mathsf{T}\mathbf{X}_c$$

where $\mathbf{X}_c$ is a centered version of the $N \times D$ design matrix.

1.1 PCA setup

Suppose we have an (unlabeled) dataset $\mathcal{D} = \{\boldsymbol{x}_n : n = 1 : N\}$, where $\boldsymbol{x}_n \in \mathbb{R}^D$. We can represent this as an $N \times D$ data matrix $\mathbf{X}$. We will assume $\overline{\boldsymbol{x}} = \frac{1}{N}\sum_{n=1}^{N} \boldsymbol{x}_n = \mathbf{0}$, which we can ensure by centering the data.

We would like to approximate each $\boldsymbol{x}_n$ by a low dimensional representation, $\boldsymbol{z}_n \in \mathbb{R}^L$. We assume that each $\boldsymbol{x}_n$ can be explained in terms of a weighted combination of basis functions $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_L$, where each $\boldsymbol{w}_k \in \mathbb{R}^D$, and where the weights are given by $\boldsymbol{z}_n \in \mathbb{R}^L$, i.e.

$$\boldsymbol{x}_n \approx \sum_{k=1}^{L} z_{nk}\boldsymbol{w}_k$$

The vector $\boldsymbol{z}_n$ is the low dimensional representation of $\boldsymbol{x}_n$, and is known as the **latent vector**, the collection of these latent variables are called the **latent factors**.

The error produced by this approximation:

$$\mathcal{L}(\mathbf{W}, \mathbf{Z}) = \frac{1}{N}\|\mathbf{X} - \mathbf{Z}\mathbf{W}^\mathsf{T}\|_F^2 = \frac{1}{N}\|\mathbf{X}^\mathsf{T} - \mathbf{W}\mathbf{Z}^\mathsf{T}\|_F^2 = \frac{1}{N}\sum_{n=1}^{N}\|\boldsymbol{x}_n - \mathbf{W}\boldsymbol{z}_n\|^2$$

where

- $\mathbf{X} \in \mathbb{R}^{N\times D}$: data matrix, rows are centered data points $\mathbf{x}_n^T$
- $\mathbf{Z} \in \mathbb{R}^{N\times L}$: latent code matrix, rows are centered data points $\mathbf{z}_n^T$
- $\mathbf{W} \in \mathbb{R}^{D\times L}$: basis matrix, columns $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_L$ are orthonormal
- approximating each $\boldsymbol{x}_n$ by $\hat{\boldsymbol{x}}_n = \mathbf{W}\boldsymbol{z}_n$; or, $\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}^T$.
- the minimization of $\mathcal{L}$ is subject to constraints $\mathbf{w}_k^T\mathbf{w_j} = 0 \,\text{for}\, k \neq j; 1 \,\text{for}\, k = j$, i.e., $\mathbf{W}$ is orthogonal

1.2 Algorithm derivation

Base case

Starting by considering the best 1d solution , $\mathbf{w}_1 \in \mathbb{R}^D$, letting the coefficients for each datapoints associated with the first basis vector be denoted by $\tilde{\mathbf{z}}_1 = [z_{11}, \ldots, z_{N1}] \in \mathbb{R}^N$. The reconstruction error is

$$\begin{aligned}
\mathcal{L}(\boldsymbol{w}_1, \tilde{\boldsymbol{z}}_1) &= \frac{1}{N}\sum_{n=1}^{N} \|\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1\|^2 = \frac{1}{N}\sum_{n=1}^{N} (\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1)^\mathsf{T}(\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1) \\
&= \frac{1}{N}\sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - 2z_{n1}\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n + z_{n1}^2\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1] \\
&= \frac{1}{N}\sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - 2z_{n1}\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n + z_{n1}^2] \quad \text{since}\, \boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1 = 1
\end{aligned}$$

Taking derivatives wrt z_{n1} and equating to zero gives

$$\frac{\partial}{\partial z_{n1}}\mathcal{L}(\boldsymbol{w}_1, \tilde{\boldsymbol{z}}_1) = \frac{1}{N}[-2\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n + 2z_{n1}] = 0 \Rightarrow z_{n1} = \boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n$$

So the optimal embedding is obtained by orthogonally projecting the data onto $\boldsymbol{w}_1$. Plugging this back in gives the loss for the weights:

$$\mathcal{L}(\boldsymbol{w}_1) = \mathcal{L}(\boldsymbol{w}_1, \tilde{\boldsymbol{z}}_1^*(\boldsymbol{w}_1)) = \frac{1}{N}\sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - z_{n1}^2] = \text{const} - \frac{1}{N}\sum_{n=1}^{N} z_{n1}^2$$

To solve for $\boldsymbol{w}_1$, note that

$$\mathcal{L}(\boldsymbol{w}_1) = -\frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 = -\frac{1}{N}\sum_{n=1}^{N} \boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}\boldsymbol{w}_1 = -\boldsymbol{w}_1^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1$$

where $\boldsymbol{\Sigma}$ is the empirical covariance matrix (since we assumed the data is centered). We can trivially optimize this by letting $\|\boldsymbol{w}_1\| \to \infty$, so we impose the constraint $\|\boldsymbol{w}_1\| = 1$ and instead optimize

$$\tilde{\mathcal{L}}(\boldsymbol{w}_1) = \boldsymbol{w}_1^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 - \lambda_1(\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1 - 1)$$

where λ_1 is a Lagrange multiplier. Taking derivatives and equating to zero we have

$$\frac{\partial}{\partial \boldsymbol{w}_1}\tilde{\mathcal{L}}(\boldsymbol{w}_1) = 2\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 - 2\lambda_1\boldsymbol{w}_1 = 0$$

$$\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 = \lambda_1\boldsymbol{w}_1$$

Hence the optimal direction onto which we should project the data is an eigenvector of the covariance matrix. Left multiplying by $\boldsymbol{w}_1^\mathsf{T}$ (and using $\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1 = 1$) we find

$$\boldsymbol{w}_1^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 = \lambda_1$$

Since we want to maximize this quantity (minimize the loss), we pick the eigenvector which corresponds to the largest eigenvalue.

Induction Step

For another direction $\mathbf{w}_2$, the reconstructon error is

$$\mathcal{L}(\boldsymbol{w}_1, \tilde{\boldsymbol{z}}_1, \boldsymbol{w}_2, \tilde{\boldsymbol{z}}_2) = \frac{1}{N}\sum_{n=1}^{N} \|\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1 - z_{n2}\boldsymbol{w}_2\|^2$$

Optimizing wrt $\boldsymbol{w}_1$ and $\boldsymbol{z}_1$ gives the same solution as before. $\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_2} = 0$ yields $z_{n2} = \boldsymbol{w}_2^\mathsf{T}\boldsymbol{x}_n$. Substituting in yields

$$\mathcal{L}(\boldsymbol{w}_2) = \frac{1}{N}\sum_{n=1}^{N}[\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - \boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}\boldsymbol{w}_1 - \boldsymbol{w}_2^\mathsf{T}\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}\boldsymbol{w}_2] = \text{const} - \boldsymbol{w}_2^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_2$$

Dropping the constant term, plugging in the optimal $\boldsymbol{w}_1$ and adding the constraints yields

$$\tilde{\mathcal{L}}(\boldsymbol{w}_2) = -\boldsymbol{w}_2^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_2 + \lambda_2(\boldsymbol{w}_2^\mathsf{T}\boldsymbol{w}_2 - 1) + \lambda_{12}(\boldsymbol{w}_2^\mathsf{T}\boldsymbol{w}_1 - 0)$$

with the extra constraint term, it can be shown that the solution is given by the eigenvector with the second largest eigenvalue:

$$\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_2 = \lambda_2\boldsymbol{w}_2$$

The proof continues in this way to show that $\hat{\mathbf{W}} = \mathbf{U}_L$.

1.3 Computational tricks

1. **High-dimensional data.** PCA finds the eigenvectors of the $D \times D$ covariance matrix $\mathbf{X}^\mathsf{T}\mathbf{X}$. If $D > N$, it is faster to work with the $N \times N$ matrix $\mathbf{X}\mathbf{X}^\mathsf{T}$. Let $\mathbf{U}$ be an orthonormal matrix containing the eigenvectors of $\mathbf{X}\mathbf{X}^\mathsf{T}$ with corresponding eigenvalues in $\boldsymbol{\Lambda}$. By definition we have $(\mathbf{X}\mathbf{X}^\mathsf{T})\mathbf{U} = \mathbf{U}\boldsymbol{\Lambda}$. Pre-multiplying by $\mathbf{X}^\mathsf{T}$ gives

   $$(\mathbf{X}^\mathsf{T}\mathbf{X})(\mathbf{X}^\mathsf{T}\mathbf{U}) = (\mathbf{X}^\mathsf{T}\mathbf{U})\boldsymbol{\Lambda}$$

   from which we see that the eigenvectors of $\mathbf{X}^\mathsf{T}\mathbf{X}$ are $\mathbf{V} = \mathbf{X}^\mathsf{T}\mathbf{U}$, with eigenvalues $\boldsymbol{\Lambda}$ as before. However, these eigenvectors are not normalized, since $\|\boldsymbol{v}_j\|^2 = \boldsymbol{u}_j^\mathsf{T}\mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{u}_j = \lambda_j\boldsymbol{u}_j^\mathsf{T}\boldsymbol{u}_j = \lambda_j$. The normalized eigenvectors are given by

   $$\mathbf{V} = \mathbf{X}^\mathsf{T}\mathbf{U}\boldsymbol{\Lambda}^{-\frac{1}{2}}$$

2. **Computing PCA using SVD.** Let $\mathbf{U}_\Sigma\boldsymbol{\Lambda}_\Sigma\mathbf{U}_\Sigma^\mathsf{T}$ be the top L eigendecomposition of the covariance matrix $\boldsymbol{\Sigma} \propto \mathbf{X}^\mathsf{T}\mathbf{X}$ (assume $\mathbf{X}$ is centered). Recall the optimal estimate of the projection weights $\mathbf{W}$ is given by the top L eigenvalues, so $\mathbf{W} = \mathbf{U}_\Sigma$. Now let $\mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\mathsf{T} \approx \mathbf{X}$ be the L-truncated SVD approximation to the data matrix $\mathbf{X}$. By SVD definition, the columns of $\mathbf{V}_X$ are the eigenvectors of $\mathbf{X}^\mathsf{T}\mathbf{X}$, so

   $$\mathbf{V}_X = \mathbf{U}_\Sigma = \mathbf{W}.$$

   In addition, the eigenvalues of the covariance matrix are related to the singular values of the data matrix via $\lambda_k = s_k^2/N$. Now suppose we are interested in the principal components (or PC scores), rather than the projection matrix. We have

   $$\mathbf{Z} = \mathbf{X}\mathbf{W} = \mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\mathsf{T}\mathbf{V}_X = \mathbf{U}_X\mathbf{S}_X$$

   Finally, if we want to approximately reconstruct the data, we have

   $$\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}^\mathsf{T} = \mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\mathsf{T}$$

   This is precisely the same as a **truncated SVD approximation**.

Thus, PCA can be performed either using an eigendecomposition of $\boldsymbol{\Sigma}$ or an SVD decomposition of $\mathbf{X}$. The latter is often preferable, for computational reasons.

2 Generalized Linear Models

A GLM is a conditional version of an exponential family distribution $p(y \mid \theta) = h(y) \exp\left(\theta^\top T(y) - A(\theta)\right)$, in which the natural parameters are a linear function of the input. More precisely, the model has the following form:

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}, \sigma^2) = \exp\left[\frac{y_n \eta_n - A(\eta_n)}{\sigma^2} + \log h(y_n, \sigma^2)\right]$$

where

- y_n is the output/response variable, $\mathbf{x_n}$ is the input/feature vector, $\mathbf{w}$ and σ is the learnable model parameter
- $\eta_n \triangleq \mathbf{w}^\mathsf{T}\mathbf{x}_n$ is the (input dependent) natural parameter or canonical parameter
- $A(\eta_n)$ is the log normalizer
- $\mathcal{T}(y_n) = y_n$ is the sufficient statistic, which is the smallest summary of the output data y that retains everything relevant for estimating the parameter θ – while it may still lose other information about y itself.
- σ^2 is the dispersion term
- $h(y_n, \sigma^2)$ is the scaling constant or base measure

We can show that the mean and variance of the response variable are as follows:

$$\int p(y_n \mid \eta_n, \sigma^2)\, dy_n = 1 \quad \text{for all } \eta_n$$

To derive the mean, differentiate the equation w.r.t to η_n

$$\int \frac{\partial p(y_n \mid \eta_n, \sigma^2)}{\partial \eta_n}\, dy_n = 0$$

$$\int p(y_n \mid \eta_n, \sigma^2) \cdot \frac{y_n - A'(\eta_n)}{\sigma^2}\, dy_n = 0$$

$$\frac{1}{\sigma^2}\left[\int y_n\, p(y_n \mid \eta_n, \sigma^2)\, dy_n - A'(\eta_n)\right] = 0$$

thus, the **mean** is:

$$\mathbb{E}\left[y_n \mid \mathbf{x}_n, \mathbf{w}, \sigma^2\right] = A'(\eta_n) \triangleq \ell^{-1}(\eta_n)$$

where the mapping from the linear inputs to the mean of the output using $\mu_n = \ell^{-1}(\eta_n)$, and the function ℓ is known as the link function, and ℓ^{-1} is known as the mean function.

To derive the variance, differentiate the above equation w.r.t η_n again,

$$\int \left[-A''(\eta_n) \cdot p + (y_n - A'(\eta_n)) \cdot p \cdot \frac{y_n - A'(\eta_n)}{\sigma^2}\right] dy_n = 0$$

$$-A''(\eta_n) \int p\, dy_n + \frac{1}{\sigma^2} \int (y_n - A'(\eta_n))^2\, p\, dy_n = 0$$

thus, the **variance** is:

$$\mathbb{V}\left[y_n \mid \mathbf{x}_n, \mathbf{w}, \sigma^2\right] = A''(\eta_n)\sigma^2$$

2.1 Logistic regression

Binary logistic regression is used when $y_n \in \{0, 1\}$. It has the form

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \mu_n^{y_n}(1 - \mu_n)^{1-y_n}$$

where

$$\mu_n = \sigma(\eta_n) = \frac{1}{1 + \exp(-\eta_n)}, \quad \eta_n = \mathbf{w}^\mathsf{T}\mathbf{x}_n.$$

Hence

$$\log p(y_n \mid \mathbf{x}_n, \mathbf{w}) = y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)$$

Using $\mu_n = \exp(\eta_n)/(1 + \exp(\eta_n))$, in GLM form:

$$\log p(y_n \mid \mathbf{x}_n, \mathbf{w}) = y_n\eta_n - \log(1 + \exp(\eta_n))$$

We see that $A(\eta_n) = \log(1 + \exp(\eta_n))$ and hence

$$\mathbb{E}[y_n] = A'(\eta_n) = \frac{1}{1 + \exp(-\eta_n)}, \quad \mathbb{V}[y_n] = A''(\eta_n) = \mu_n(1 - \mu_n)$$

Thus the link function is the logit function, $\ell(\mu_n) = \log\left(\frac{\mu_n}{1-\mu_n}\right)$, and the mean function is the sigmoid function, $\ell^{-1}(\eta_n) = \sigma(\eta_n)$.

2.2 Linear regression

Linear regression has the form with $y_n \in \mathbb{R}$

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n)^2\right)$$

Hence

$$\log p(y_n \mid \mathbf{x}_n, \mathbf{w}, \sigma^2) = -\frac{1}{2\sigma^2}(y_n - \eta_n)^2 - \frac{1}{2}\log(2\pi\sigma^2)$$

where $\eta_n = \mathbf{w}^\mathsf{T}\mathbf{x}_n$. In GLM form:

$$\log p(y_n \mid \mathbf{x}_n, \mathbf{w}, \sigma^2) = \frac{y_n\eta_n - \frac{\eta_n^2}{2}}{\sigma^2} - \frac{1}{2}\left(\frac{y_n^2}{\sigma^2} + \log(2\pi\sigma^2)\right)$$

We see that $A(\eta_n) = \eta_n^2/2$ and hence

$$\mathbb{E}[y_n] = \eta_n = \mathbf{w}^\mathsf{T}\mathbf{x}_n, \quad \mathbb{V}[y_n] = \sigma^2$$

2.3 Binomial regression

If the response variable is the number of successes in N_n trials, $y_n \in \{0, \ldots, N_n\}$, we can use binomial regression, which is defined by

$$p(y_n \mid \mathbf{x}_n, N_n, \mathbf{w}) = \mathrm{Bin}(y_n \mid \sigma(\mathbf{w}^\mathsf{T}\mathbf{x}_n), N_n)$$

note that binary logistic regression is the special case when $N_n = 1$.

The log pdf is given by

$$\log p(y_n \mid \mathbf{x}_n, N_n, \mathbf{w}) = y_n \log \mu_n + (N_n - y_n) \log(1 - \mu_n) + \log \binom{N_n}{y_n}$$

$$= y_n \log \left(\frac{\mu_n}{1 - \mu_n} \right) + N_n \log(1 - \mu_n) + \log \binom{N_n}{y_n}$$

where $\mu_n = \sigma(\eta_n)$. To rewrite this in GLM form, let us define

$$\eta_n \triangleq \log \left(\frac{\mu_n}{1 - \mu_n} \right) = \log \left[\frac{1}{1 + e^{-\mathbf{w}^\mathsf{T} \mathbf{x}_n}} \frac{1 + e^{-\mathbf{w}^\mathsf{T} \mathbf{x}_n}}{e^{-\mathbf{w}^\mathsf{T} \mathbf{x}_n}} \right] = \log \left(\frac{1}{e^{-\mathbf{w}^\mathsf{T} \mathbf{x}_n}} \right) = \mathbf{w}^\mathsf{T} \mathbf{x}_n$$

Hence we can write binomial regression in GLM form as follows:

$$\log p(y_n \mid \mathbf{x}_n, N_n, \mathbf{w}) = y_n \eta_n - A(\eta_n) + h(y_n)$$

where

$$h(y_n) = \log \binom{N_n}{y_n}, \quad A(\eta_n) = -N_n \log(1 - \mu_n) = N_n \log(1 + e^{\eta_n})$$

Hence,

$$\mathbb{E}[y_n] = \frac{dA}{d\eta_n} = \frac{N_n e^{\eta_n}}{1 + e^{\eta_n}} = N_n \mu_n, \quad \mathbb{V}[y_n] = \frac{d^2 A}{d\eta_n^2} = N_n \mu_n (1 - \mu_n)$$

2.4 Poisson regression

If the response variable is an integer count, $y_n \in \{0, 1, \ldots\}$, we can use Poisson regression, which is defined by

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \text{Poi}(y_n \mid \exp(\mathbf{w}^\mathsf{T} \mathbf{x}_n))$$

where

$$\text{Poi}(y \mid \mu) = e^{-\mu} \frac{\mu^y}{y!}$$

is the Poisson distribution.

The log pdf is given by

$$\log p(y_n \mid \mathbf{x}_n, \mathbf{w}) = y_n \log \mu_n - \mu_n - \log(y_n!)$$

where $\mu_n = \exp(\mathbf{w}^\mathsf{T} \mathbf{x}_n)$. Hence in GLM form we have

$$\log p(y_n \mid \mathbf{x}_n, \mathbf{w}) = y_n \eta_n - A(\eta_n) + h(y_n)$$

where $\eta_n = \log(\mu_n) = \mathbf{w}^\mathsf{T} \mathbf{x}_n$, $A(\eta_n) = \mu_n = e^{\eta_n}$, and $h(y_n) = -\log(y_n!)$. Hence

$$\mathbb{E}[y_n] = \frac{dA}{d\eta_n} = e^{\eta_n} = \mu_n, \quad \mathbb{V}[y_n] = \frac{d^2 A}{d\eta_n^2} = e^{\eta_n} = \mu_n$$

2.5 Canonical and non-canonical link functions

Link functions connect three quantities:

- θ_n — the **natural parameter** of the *exponential family distribution*.
- $\mu_n = \mathbb{E}[y_n]$ — the **mean** of the output distribution.
- $\eta_n = \boldsymbol{w}^\top \boldsymbol{x}_n$ — the **linear predictor**, built from the *inputs* and weights.

The two connecting functions:

$$\theta_n \xrightarrow{A'} \mu_n \xrightarrow{\ell} \eta_n$$

- A' (derivative of the log-partition function) always maps $\theta_n \to \mu_n$. This relationship is fixed by the choice of exponential-family distribution (Bernoulli, Gaussian, Poisson, …) and never changes.
- ℓ, the **link function**, maps $\mu_n \to \eta_n$. This is the function *you choose* when designing the GLM. Its inverse, ℓ^{-1}, is the **mean function**: $\mu_n = \ell^{-1}(\eta_n)$ — the direction actually used to generate predictions from η_n.

Canonical link. Choosing $\ell = (A')^{-1}$ makes the link exactly undo A':

$$\eta_n = g(\mu_n) = (A')^{-1}(A'(\theta_n)) = \theta_n$$

so the natural parameter and linear predictor coincide: $\theta_n = \eta_n$.

Non-canonical link. Choosing any $g \neq (A')^{-1}$ gives

$$\eta_n = g(A'(\theta_n)) \neq \theta_n \quad \text{in general.}$$

The mean is still recovered correctly via $\mu_n = g^{-1}(\eta_n)$, but the natural parameter θ_n is no longer equal to the linear predictor η_n—recovering it requires the separate map $(A')^{-1}(\mu_n)$.

Examples of links:

- Logit: canonical link $\theta_n = \ell(\mu_n) = \log \frac{\mu_n}{1-\mu_n}$.
- Complementary log-log: non-canonical link $\eta_n = \ell(\mu_n) = \log(-\log(1-\mu_n))$

2.6 Maximum likelihood estimation

GLMs can be fit the negative log-likelihood as the objective function (ignoring constant term h):

$$\text{NLL}(\mathbf{w}) = -\log p(\mathcal{D} \mid \mathbf{w}) = -\frac{1}{\sigma^2}\sum_{n=1}^{N} \ell_n = -\frac{1}{\sigma^2}\sum_{n=1}^{N}[\eta_n y_n - A(\eta_n)]$$

where $\eta_n = \mathbf{w}^\mathsf{T}\mathbf{x}_n$.

To find the MLE, we must solve $\nabla\text{NNL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) = 0$. For notational simplicity, we will assume $\sigma^2 = 1$. The gradient for a single term is:

$$\begin{aligned}\mathbf{g}_n &= -\frac{1}{\sigma^2}\frac{\partial \ell_n}{\partial \mathbf{w}} = -\frac{1}{\sigma^2}\frac{\partial \ell_n}{\partial \eta_n}\frac{\partial \eta_n}{\partial \mathbf{w}} \\ &= -\frac{1}{\sigma^2}\left(y_n - A'(\eta_n)\right)\mathbf{x}_n = -\frac{1}{\sigma^2}(y_n - \mu_n)\mathbf{x}_n\end{aligned}$$

where $\mu_n = f(\mathbf{w}^\mathsf{T}\mathbf{x})$, and f is the inverse link function that maps from canonical parameters to mean parameters. If we interpret $(y_n - \mu_n)$ as an error signal, then the gradient $-\frac{1}{\sigma^2}\sum_{n=1}^{N}(y_n - \mu_n)\mathbf{x}_n$ weights each input $\mathbf{x}_n$ by its error, and averages the result.

Gradient-based optimization will find a stationary points where $\mathbf{g}(\mathbf{w}) = 0$, we need to examine the Hessian matrix to determine whether it is the global optimum. The Hessian is given by

$$\begin{aligned}\mathbf{H} &= \frac{\partial^2}{\partial \boldsymbol{w}\partial \boldsymbol{w}^\top}\mathrm{NLL}(\boldsymbol{w}) \\ &= -\sum_{n=1}^{N}\frac{\partial \boldsymbol{g}_n}{\partial \boldsymbol{w}^\top} = -\sum_{n=1}^{N}\frac{\partial \boldsymbol{g}_n}{\partial \mu_n}\frac{\partial \mu_n}{\partial \boldsymbol{w}^\top} = -\sum_{n=1}^{N}\boldsymbol{x}_n f'(\boldsymbol{w}^\top \boldsymbol{x}_n)\boldsymbol{x}_n^\top\end{aligned}$$

In general, the Hessian is positive definite, since $f'(\boldsymbol{w}^\top \boldsymbol{x}_n) > 0$, hence the negative log-likelihood is convex, so the MLE for a GLM is unique given $f(\boldsymbol{w}^\top \boldsymbol{x}_n) > 0$ for all n.

Based on these results, gradient-based optimizers can be used to estimate the model weights.

2.6.1 Stochastic gradient descent

The goal is to solve

$$\hat{\boldsymbol{w}} \triangleq \operatorname*{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = \operatorname*{argmin}_{\boldsymbol{w}} \mathrm{NLL}(\mathbf{w})$$

One of the methods is stochastic gradient descent: we use a minibatch from the whole dataset of size N, then we get the following update equation:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla_{\mathbf{w}} \mathrm{NLL}(\mathbf{w}_t) = \mathbf{w}_t - \alpha_t(\mu_n - y_n)\mathbf{x}_n$$

where we replaced the average over all N samples with a stochastically chosen minibatch size at each step t.

Since we know the objective is convex, then it can be shown that this will converge to a global optimum given the learning rate α_t is decayed appropriately at each step t.